

An Instruction Is Just a Number — Watching a CPU Think

CM-203 Computer Organization · Capstone one-pager · Andrei Voicu · July 2026

PROBLEM STATEMENT / OBJECTIVE

Every computer runs one loop — fetch, decode, execute — billions of times a second, invisibly. This capstone builds a complete Von Neumann machine in software — unified memory, registers, a program counter, an ALU, and my own instruction set — and slows it to *one heartbeat at a time*, so the audience watches each phase happen live. Then it proves the central claim of the architecture — an instruction and a number are the *same bits in the same memory*, distinguished only by the lens — first by reading one word both ways, finally with a program that rewrites its own instructions while running.

APPROACH AND TECHNICAL SETUP / ENVIRONMENT

The machine is ~500 lines of dependency-free C99 (Windows, Linux/WSL2, and an Arduino UNO Q's Linux side): 256 × 16-bit words of memory holding *both* program and data, registers R0–R3, an 8-bit PC that wraps like a real address bus, an instruction register, and a zero flag. Instructions are fixed-width — [opcode:4 | reg:4 | operand:8] — every field one hex digit, so any word can be hand-assembled on stage, and the 8-bit operand names exactly the 256 memory slots, by design. The terminal interface shows every word through *both lenses at once* (as a number and as an instruction), highlights the PC's slot and the most recently rewritten slot, and narrates each phase in plain English; execution is deterministic, so the live demo is exactly reproducible. The demonstrations:

- **The heartbeat** — one keypress per phase: fetch ($IR \leftarrow mem[PC]$, PC advances), decode (the word splits into fields), execute (the ALU fires, a register changes).
- **A loop that decides** — sum 1..N: SUB drives the zero flag, JNZ throws the PC backwards until the flag flips — the branch visibly taken, then not taken, then 55 appears.
- **The reveal** — the word just executed as ADD R0, R1 is printed as plain data: 8193. Identical bits (0x2001); only the lens differs.
- **Being the assembler** — encode ADD R0, R1 live, field by field, into 0x2001, type the number straight into memory, and run it: the program was never magic, it's bits.
- **The program that writes its own ending** — as loaded it can never stop: its last word jumps back to the start. Mid-run it STORES the number 0 over that slot — and 0, read as an instruction, is HALT. When the PC arrives, the machine executes the program's own edit. Only a shared instruction/data memory allows this; real CPUs forbid it (W^X, instruction caches).
- **A physical heartbeat** (stretch) — the Arduino UNO Q blinks one LED per phase, driven by the emulator's phase hook: the invisible loop, visible across the room.

The machine is already built and working: both demo programs run end to end, in a public repo whose commit history doubles as the lab notebook (github.com/Voicu-Andrei/cm203-capstone).

TOPICS FROM "COMPUTER ORGANIZATION" ADDRESSED

Core (Ch. 5): the Von Neumann model, fetch–decode–execute, registers, PC and IR, ALU and flags, unified memory. Connected areas: Ch. 1–3 — the whole machine is one C struct and one array, the memory model made literal; Ch. 4 — the word type is uint16_t, so two's-complement wraparound is real, and "same bits, different lens" is the thesis itself; Ch. 6–10 — designing an ISA and hand-assembling it, with real parallels: my CMP is x86's CMP (a subtraction that discards its result) and the fixed-width, twelve-instruction set is RISC-shaped, like ARM64. Deliberately abstracted: pipelines, caches, gates, timing.

RELEVANT OUTCOMES / WHY THIS IS INTERESTING

The invisible machinery becomes watchable: why an instruction and a number are the same thing, what a branch physically is, and why self-modifying code is trivial here yet forbidden in real CPUs — the power and the danger of the Von Neumann model. Every register, instruction, and width has a reason I can defend — depth over breadth, live and reproducible.